A Minimalist Datatype-Generic Putback-Based Bidirectional Programming Language

Hsiang-Shang Ko[2], Tao Zan[1], and Zhenjiang Hu[1,2]

[1] SOKENDAI (The Graduate University for Advanced Studies), Japan
[2] National Institute of Informatics, Japan

1 Introduction

[1]

2 Decidable partiality

By "decidable partiality" we mean that the transformations are actually total functions which wrap their results in the standard Maybe datatype:

```
data Maybe (A : Set) : Set where
  just    : A → Maybe A
  nothing : Maybe A
```

This is different from the usual partial functions as formalised in terms of complete partial orders: In Section 4 we will need to define programs that produce some result or fail respectively when their input fails or produces some result, but this is not possible in the setting of complete partial orders, since such functions violate monotonicity, a necessary condition for computability.

```
data Par : Set → Set₁ where
  return        : {A : Set} → A → Par A
  _>>=_         : {A B : Set} → Par A → (A → Par B) → Par B
  fail          : {A : Set} → Par A
  catch         : {A B : Set} → Par A → (A → Par B) → Par B → Par B
  assert_then_  : {A : Set} → Bool → Par A → Par A

runPar : {A : Set} → Par A → Maybe A
runPar (return x          ) = just x
runPar (mx >>= f          ) with runPar mx
runPar (mx >>= f          ) | just x    = runPar (f x)
runPar (mx >>= f          ) | nothing = nothing
runPar fail                   = nothing
runPar (catch mx f my     ) with runPar mx
runPar (catch mx f my     ) | just x    = runPar (f x)
runPar (catch mx f my     ) | nothing = runPar my
runPar (assert true then mx) = runPar mx
runPar (assert false then mx) = nothing
```

```
mutual
  data CompSeq : {A : Set} → Par A → A → Set₁ where
    ret           : {A : Set} {x x′ : A} → x ≡ x′ → CompSeq (return x) x′
    bind          : {A B : Set} {x : A} {mx : Par A} {f : A → Par B} {y : B} →
                    CompSeq mx x → CompSeq (f x) y → CompSeq (mx >>= f) y
    catch-fst     : {A B : Set} {mx : Par A} {f : A → Par B} {my : Par B} {x : A} {z : B} →
                    CompSeq mx x → CompSeq (f x) z → CompSeq (catch mx f my) z
    catch-snd     : {A B : Set} {mx : Par A} {f : A → Par B} {my : Par B} {z : B} →
                    FailedCompSeq mx → CompSeq my z → CompSeq (catch mx f my) z
    assert-then   : {A : Set} {b : Bool} {mx : Par A} {x : A} → b ≡ true → CompSeq mx x → CompSeq (assert
  data FailedCompSeq : {A : Set} → Par A → Set₁ where
    bind-fst      : {A B : Set} {mx : Par A} {f : A → Par B} → FailedCompSeq mx → FailedCompSeq (mx >>= f
    bind-snd      : {A B : Set} {mx : Par A} {x : A} {f : A → Par B} →
                    CompSeq mx x → FailedCompSeq (f x) → FailedCompSeq (mx >>= f)
    fail          : {A : Set} → FailedCompSeq (fail {A})
    catch-fst     : {A B : Set} {mx : Par A} {f : A → Par B} {my : Par B} →
                    FailedCompSeq mx → FailedCompSeq my → FailedCompSeq (catch mx f my)
    catch-snd     : {A B : Set} {mx : Par A} {f : A → Par B} {my : Par B} {x : A} →
                    CompSeq mx x → FailedCompSeq (f x) → FailedCompSeq (catch mx f my)
    assert-fst    : {A : Set} {mx : Par A} {b : Bool} → b ≡ false → FailedCompSeq (assert b then mx)
    assert-snd    : {A : Set} {mx : Par A} {b : Bool} → b ≡ true → FailedCompSeq mx → FailedCompSeq (assert b
```

Fig. 1. Definitions of *CompSeq* and *FailedCompSeq*.

3 Putback-based bidirectional transformations

$$
\begin{array}{ll}
 & get\ s \mapsto v \\
\Rightarrow & \{\,\textsc{GetPut} \text{ for } get\,\} \\
 & put\ s\ v \mapsto s \\
\Rightarrow & \{\,\textsc{PutGet} \text{ for } get'\,\} \\
 & get'\ s \mapsto v
\end{array}
$$

4 BiGUL

5 Reduction of common features

6 Discussion

Haskell implementation

References

1. J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, 29(3):17, 2007.